Parallel Computer Architecture & Programming
Problem Statement 5

Scalable Parallel K-Means Clustering Using MPI and CUDA

Mini-Project Report

This report compares three implementations of K-means on a synthetic dataset containing 100,000 points and 16 features. The serial CPU program provides the reference result. The MPI version divides the points between two ranks, while the hybrid version uses MPI for distributed work and CUDA for the local point-assignment step.

Field	Detail
Course	Parallel Computer Architecture & Programming (PS-5)
Problem statement	Scalable parallel clustering for large point sets
Project title	Scalable Parallel K-Means Clustering Using MPI and CUDA

Team

1. Tirth Miteshkumar Patel Reg. 240905272 Roll 27 CSE D
Distributed-memory MPI design, dataset partitioning, synchronization, scaling analysis, and the shared numerical core.

2. Aarush Arora Reg. 240905266 Roll 26 CSE D
CUDA assignment kernel, device-side reduction, CPU fallback when no GPU is present.

3. Udit Asthana Reg. 240905310 Roll 31 CSE D
Integration of the MPI driver and the CUDA kernel into `kmeans_hybrid`: the shared assignment contract, the `USE_CUDA_KERNEL` build, and the runners that validate the bound binary against the serial reference.

Contents

I Abstract 3

II Keywords 3

III Introduction 3

IV Literature Review 3

V Problem Statement 4

VI Objectives 4

VII Proposed Methodology 4

VIII System Architecture 4
- VIII-A MPI Layer . . . 4
- VIII-B CUDA Kernel . . . 5
- VIII-C Integration Contract . . . 5
- VIII-D Shared Header . . . 5

IX Algorithms 5
- IX-A Lloyd Iteration . . . 5
- IX-B Initialization . . . 6

X Experimental Setup 6
- X-A Workload and execution modes . . . 6
- X-B Timing and communication . . . 6
- X-C Candidate datasets . . . 6
- X-D Correctness and validation . . . 6

XI Results 7
- XI-A Execution screenshots . . . 7

XII Performance Analysis 7
- XII-A Strong scaling . . . 8
- XII-B Weak scaling . . . 8
- XII-C Limitations affecting performance . . . 8

XIII Comparison with Existing Methods 8

XIV Conclusion 8

XV Future Scope 9

References 9

I. Abstract

K-means divides a dataset into K clusters by repeatedly assigning each point to its nearest centroid and recalculating the centroids. The assignment phase requires $O(NKD)$ distance work per iteration, so it is the main target for parallel execution.

This project implements three versions of the algorithm: serial CPU, MPI CPU, and hybrid MPI+CUDA. The MPI program partitions the points across ranks and combines local cluster sums and counts with `MPI_Allreduce`. In the hybrid version, a CUDA kernel performs the local assignment work when a usable GPU is available; the program falls back to the CPU path when it is not.

The benchmark uses $N = 100{,}000$ data points, $D = 16$ features, and $K = 10$ clusters. The supplied plot summarizes 20 timing runs for each version. Mean runtime falls from 1.222 s for the serial program to 0.943 s for two-rank MPI and 0.621 s for the hybrid version. These results give mean speedups of about 1.30× and 1.97×, respectively. They describe this workload and test environment; further runs at different process counts and data sizes are needed to assess scaling.

II. Keywords

K-means clustering; high-performance computing; MPI; CUDA; distributed-memory parallelism; GPU acceleration; speedup; parallel efficiency; execution-time analysis.

III. Introduction

Each Lloyd iteration of K-means has two main stages. First, every point is assigned to the centroid with the smallest squared Euclidean distance. Second, each non-empty cluster's centroid is replaced by the mean of its assigned points. The assignment stage costs $O(NKD)$ per iteration, whereas the centroid update works on only K clusters with D coordinates each.

Point assignments are independent within an iteration, which makes them suitable for parallel execution. In the MPI version, each rank works on a separate block of points. The ranks then combine their local cluster sums and counts through a collective reduction. Since the reduction exchanges centroid statistics rather than the full point matrix, its data volume depends on K and D, not directly on N.

The hybrid version applies the same idea within each rank: a CUDA kernel assigns local points to centroids, while MPI coordinates the distributed update. The measured runtime includes the work needed to allocate device buffers and transfer data, so the results reflect the full implementation rather than kernel execution alone.

IV. Literature Review

MacQueen's early work described a method for classifying multivariate observations through repeated reassignment [1]. Lloyd later formalized the iterative least-squares procedure commonly used for K-means [2]. Both works underpin the assignment-and-update loop used in this project.

Distributed implementations divide the dataset among processes and combine their local statistics to update centroids. Dhillon and Modha examined clustering on distributed-memory multiprocessors and showed how this approach can be used for larger workloads [3]. The MPI implementation here follows the same general pattern: ranks retain their own points and exchange the sums and counts needed for the next iteration.

GPU-based K-means implementations parallelize the distance calculations for different points. Farivar et al. and Li et al. studied approaches to GPU execution and its associated performance costs [4], [5]. The hybrid version in this report uses CUDA for the assignment step and MPI for distributing points and combining cluster statistics. Results from those papers are useful context, but their hardware and test data differ from the present experiment, so their reported runtimes are not treated as a numerical baseline.

The implementation also supports K-means++ initialization, which chooses starting centroids with probabilities based on each point's distance from the selected centers [6]. For correctness checks, centroid labels are matched by minimum cost because two valid K-means runs can number the same clusters differently.

V. Problem Statement

Problem Statement 5 requires a K-means program that can be run in serial, across MPI processes, and with GPU acceleration. The three versions need to use the same data format and centroid-update rule so that their results can be compared on a common basis.

The implementation follows four practical requirements:

- The serial, MPI, and hybrid programs read the same binary input format.
- Rank 0 reads the complete point matrix and distributes contiguous blocks to the other ranks.
- Each iteration records computation and communication timing separately.
- The CUDA assignment routine follows the CPU routine's interface, allowing a CPU fallback when a usable GPU is unavailable.

The default stopping condition uses the largest centroid movement between consecutive iterations:

$$\Delta = \max_{k:\, n_k > 0} \left\| \mu_k^{\text{new}} - \mu_k^{\text{old}} \right\|_2 < \tau,$$

where $\tau = 10^{-4}$ and the iteration limit is 100.

VI. Objectives

The work was organized around the following objectives:

1) Build serial CPU, MPI CPU, and hybrid MPI+CUDA versions around a shared numerical implementation.
2) Test the versions on a synthetic workload with $N = 100{,}000$, $D = 16$, and $K = 10$.
3) Summarize 20 runtime measurements per mode using mean and median values.
4) Calculate speedup against the serial baseline and estimate two-rank MPI efficiency.
5) Check that parallel runs produce comparable SSE values and centroid sets.
6) Present the measurements in plots and tables, with execution screenshots added from actual runs.

VII. Proposed Methodology

The benchmark compares the three implementations on a synthetic matrix with 100,000 points and 16 features, using 10 clusters. The supplied performance plot contains 20 timing runs for each mode. Mean and median runtime are used because the mean reflects overall time while the median is less affected by occasional slow runs.

The serial CPU version provides the reference. The MPI CPU version divides the points between two ranks, assigns each rank's local points, and reduces the cluster sums and counts. In the hybrid version, the same distributed iteration calls a CUDA kernel for local point assignment when a compatible device is available. The measured hybrid time includes memory allocation, data transfers, kernel execution, synchronization, and copying results back to the host.

Correctness checks are kept separate from timing. The validation script compares the SSE objective and matches candidate centroids to the reference set using a minimum-cost assignment. This allows cluster identifiers to differ while still checking whether the resulting centroid sets agree. For reproducibility, raw results should also record the CPU and GPU models, compiler, MPI and CUDA versions, build flags, and random seed. Those details are not visible in the supplied plot and should be taken from the run logs.

VIII. System Architecture

VIII-A. MPI Layer

The file `src/kmeans_mpi.cpp` controls the distributed iteration. Rank 0 reads the matrix, broadcasts N and D, and selects the initial centroids. The point counts are divided as evenly as possible using

$$n_r = \lfloor N/P \rfloor + \mathbf{1}[r < N \bmod P], \tag{1}$$

then scattered with `MPI_Scatterv`. After initialization, rank 0 releases the full matrix. Each iteration zeros local sum and count buffers, calls `do_assignment_step`, and all-reduces three arrays: $K \times D$ double-precision coordinate sums, K integer counts, and one double-precision partial SSE. Every rank then calls `update_centroids` on the reduced globals. Labels are gathered at rank 0 with `MPI_Gatherv` after the loop.

TABLE I
Implementation variants and their assignment paths.

Binary	Build target	Assignment
`bin/kmeans_serial`	`make serial`	`assign_points_cpu` on all N points
`bin/kmeans_mpi`	`make mpi`	`assign_points_cpu` on the local block
`bin/kmeans_hybrid`	`make hybrid`	`launch_assignment_kernel` on the local block

Each rank needs the same updated centroids before the next assignment, so the cluster statistics are combined with `MPI_Allreduce` at every iteration. A separate reduction followed by a broadcast would require another collective. Once the global sums and counts are available, each rank can calculate the updated centroids locally. The implementation uses MPI collectives as defined by the MPI standard [7].

VIII-B. CUDA Kernel

The CUDA implementation in `src/cuda_assignment_kernel.cu` evaluates the assignment rule in (2). It launches one-dimensional blocks of 256 threads, with one thread responsible for each local point. A thread checks the K centroids, records the nearest one, and contributes to cluster sums and counts through `atomicAdd`. On devices without native double-precision atomic addition, the code uses a compare-and-swap loop on the 64-bit value.

The host function `launch_assignment_kernel` manages the device buffers and data transfers. For each iteration, it copies the local points and centroids to the device, launches the kernel, waits for it to finish, copies the results back, and releases the allocated memory. If no CUDA device is available or an allocation fails, the function calls `assign_points_cpu`. MPI can then continue through the same iteration sequence.

VIII-C. Integration Contract

The MPI driver uses `do_assignment_step` as the single compile-time switch controlled by `USE_CUDA_KERNEL`. The CPU and CUDA routines accept the same arguments: the local point block, its length, D, the centroids, K, labels, cluster sums, and counts. Each routine returns the local SSE.

The hybrid Makefile target compiles `src/cuda_assignment_kernel.cu` with `nvcc` and links it to the MPI driver using `-DUSE_CUDA_KERNEL` and `libcudart`. The default `make all` target builds the serial and MPI programs, so a system without `nvcc` can still build the CPU versions.

VIII-D. Shared Header

The shared header, `include/kmeans_common.h`, defines the binary format, partitioning rule, initializers, CPU assignment, centroid update, and output writers. Its functions are declared `inline`. Each input file starts with a 12-byte header containing the magic value `0x4B4D4E53` and the native 32-bit integers N and D. The header is followed by $N \times D$ row-major `float32` values. The contiguous layout can be passed directly to `MPI_Scatterv` and `cudaMemcpy`.

IX. Algorithms

IX-A. Lloyd Iteration

Let $x_i \in \mathbb{R}^D$ for $i = 1, \ldots, N$, with current centroids $\mu_1, \ldots, \mu_K$. The assignment of point i is

$$c(i) = \underset{k \in \{1,\ldots,K\}}{\arg\min} \left\| x_i - \mu_k \right\|_2^2. \tag{2}$$

The clustering objective is the within-cluster sum of squared errors (SSE),

$$\mathrm{SSE} = \sum_{i=1}^{N} \left\| x_i - \mu_{c(i)} \right\|_2^2. \tag{3}$$

Once all points have been assigned, each non-empty centroid is updated to the mean of its members:

$$\mu_k \leftarrow \frac{1}{|C_k|} \sum_{x \in C_k} x. \tag{4}$$

In the MPI version, each rank calculates its own partial sum $S_k^{(r)}$ and count $n_k^{(r)}$. A global reduction combines these values before the centroids are up-

dated:

$$S_k = \sum_{r=0}^{P-1} S_k^{(r)}, \quad n_k = \sum_{r=0}^{P-1} n_k^{(r)}, \quad \mu_k = S_k/n_k. \quad (5)$$

Cluster sums use double precision, while the stored points and centroids use single precision. The assignment step compares squared distances, so it does not need to calculate square roots.

IX-B. Initialization

The code provides two initialisation methods. With uniform sampling, it shuffles the point indices using `std::mt19937` and selects the first K rows as the initial centroids. The seed controls the shuffle.

K-means++ [6] selects the first center uniformly from the data points. Each later center is sampled with probability proportional to the squared distance from the closest center already selected:

$$\Pr(x) = \frac{D(x)^2}{\sum_{x'} D(x')^2}. \quad (6)$$

Rank 0 performs initialization and broadcasts the centroid table before the first assignment step.

X. Experimental Setup

X-A. Workload and execution modes

The benchmark uses the synthetic workload listed in Table II. The timing plot summarizes 20 runs in each mode: serial CPU, MPI CPU with two ranks, and CUDA Hybrid. The source material does not identify the host machine or software versions. Those details should be copied from the execution logs so the measurements can be reproduced on the same configuration.

X-B. Timing and communication

The timing code records the duration of `do_assignment_step` and the time spent in the three collective reductions. These measurements separate local assignment work from communication in the MPI path. At the end of the run, `MPI_MIN`, `MPI_MAX`, and `MPI_SUM` combine the per-rank totals; the metrics file reports the average, minimum, and maximum values.

The communication share for a run is calculated as

$$f_{\text{comm}} = \frac{\overline{T}_{\text{comm}}}{T_{\text{total}}}, \quad (7)$$

TABLE II
Fixed workload and benchmark configuration.

Parameter	Value
Number of data points (N)	100,000
Feature dimensions (D)	16
Number of clusters (K)	10
Number of timing runs	20 per execution mode
Baseline	Serial CPU
Distributed CPU mode	MPI CPU, 2 ranks
Accelerated mode	CUDA Hybrid (GPU+MPI)
Stopping tolerance	10^{-4} by default
Maximum iterations	100

where $\overline{T}_{\text{comm}}$ is the average per-rank communication time and T_{total} is the barrier-to-barrier wall time of the iteration loop.

For the hybrid path, the measured computation interval also contains device allocation, host-to-device transfer, kernel execution, synchronization, and the return transfer. It therefore represents the full assignment call rather than arithmetic on the GPU alone. A profiler would be needed to separate these costs.

X-C. Candidate datasets

The solver reads binary input files prepared by `scripts/generate_dataset.py` or `scripts/prepare_real_data.py`; CSV parsing is handled before the solver runs. The preparation scripts can center each feature and scale it to unit variance, which matters because Euclidean distance is affected by feature scale. The Colab notebook and default runners use a synthetic matrix of shape $10^5 \times 16$ with 10 generating clusters. The public datasets in Table III are options for additional tests; they are not the source of the runtime results reported here.

X-D. Correctness and validation

K-means converges to a local solution of (3); the result can depend on the initial centroids. With the same seed and initialization, runs are expected to agree within numerical tolerance. Exact equality is not required because cluster labels can be permuted and GPU atomic operations can accumulate values in a different order from the serial loop.

The script `scripts/validate_correctness.py`

TABLE III
Candidate datasets for additional experiments.

Dataset	Shape	Role
Synthetic	$N \times D$, configurable	Scaling, correctness, Colab demo
MNIST [8]	$70{,}000 \times 784$	High-dimensional public matrix; $K = 10$ matches digit count
KDD Cup 1999 (10%) [9]	$494{,}021 \times \sim 118$ after one-hot encoding	Mixed numeric/categorical traffic matrix
US Census 1990 [10]	$2{,}458{,}285 \times \sim 68$	Large-N throughput workload

compares a candidate output with a reference output using four checks.

1) The metrics files must agree on N, D, and K.
2) The relative SSE gap must fall within $\varepsilon = 10^{-3}$:
$$\frac{|\mathrm{SSE}_a - \mathrm{SSE}_b|}{\max(|\mathrm{SSE}_a|,\, 10^{-12})} \leq \varepsilon. \tag{8}$$
3) Centroids are paired by solving a minimum-cost assignment problem over the pairwise Euclidean-distance matrix. Each matched pair must have distance at most $\delta = 10^{-2}$.
4) When both assignment files exist, every label must lie in $\{0, \ldots, K-1\}$ and each file must contain exactly N labels.

Before writing results, each binary also checks that the cluster counts sum to N. If the count is inconsistent, the program reports the problem on standard error.

XI. Results

Table IV reports the runtime values shown in the 20-run plot. Serial CPU has a mean of 1.222 s and a median of 1.185 s. With two MPI ranks, the mean falls to 0.943 s and the median to 0.915 s. CUDA Hybrid records 0.621 s for both measures.

CUDA Hybrid is the fastest mode in this experiment. Its mean runtime is about 49.2% lower than the serial result, while two-rank MPI reduces the mean by about 22.8%. These percentages apply to the stated dataset and test environment; they should not be assumed to hold for other problem sizes or hardware.

XI-A. Execution screenshots

The benchmark plot was provided, but terminal screenshots for the three program runs were not available in the materials used to prepare this report. Replace each placeholder below with a screenshot captured from the corresponding run before submission. The source file will include images saved under the filenames shown in the panels.

XII. Performance Analysis

Speedup is the serial runtime divided by the runtime of the selected parallel version. For mean runtimes, it is
$$S_{\mathrm{mean}} = \frac{T_{\mathrm{serial,mean}}}{T_{\mathrm{mode,mean}}}. \tag{9}$$
For two-rank MPI, the conventional efficiency estimate is $E = S/P$, with $P = 2$. The mean values give $S = 1.222/0.943 \approx 1.297$, so the estimated efficiency is $1.297/2 \approx 64.8\%$. The median values give $1.185/0.915 \approx 1.295$, which leads to nearly the same estimate. Since these calculations use end-to-end runtime, communication and other implementation costs are included.

The hybrid result is reported as speedup rather than rank-based efficiency. A CPU rank and a GPU are not equivalent processing units, and the graph does not provide enough information to define a comparable processor count. The end-to-end speedup against the serial program remains meaningful for this workload.

The violin plot places most CUDA Hybrid runs close to 0.62 s. The CPU distributions are wider, and the serial results include a small number of slower runs. The figure does not provide the raw timings, so standard deviation, confidence intervals, and exact

TABLE IV
OBSERVED RUNTIME AND SPEEDUP FOR $N = 100{,}000$, $D = 16$, AND $K = 10$. SPEEDUPS ARE RELATIVE TO SERIAL CPU.

Execution mode	Mean (s)	Median (s)	Mean S	Median S	Mean reduction
Serial CPU	1.222	1.185	1.00×	1.00×	–
MPI CPU (2 ranks)	0.943	0.915	1.30×	1.30×	22.8%
CUDA Hybrid (GPU+MPI)	0.621	0.621	1.97×	1.91×	49.2%

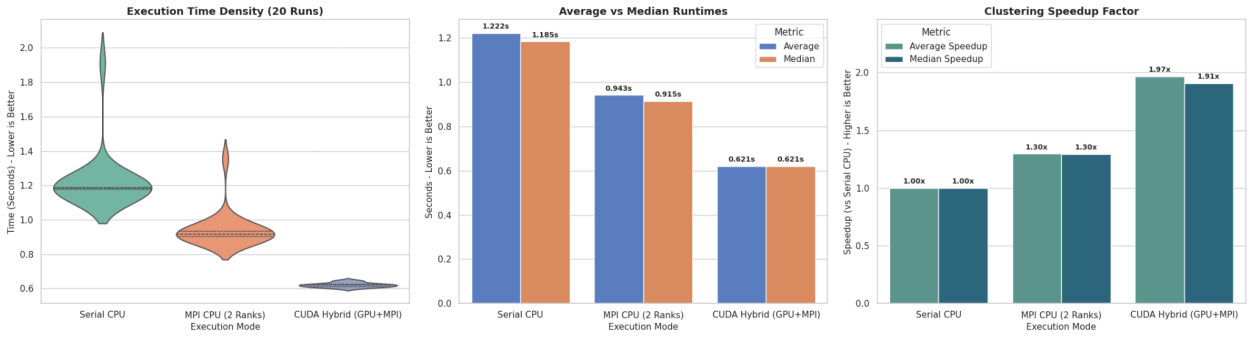


Fig. 1. Runtime distributions over 20 runs, mean and median runtimes, and speedup relative to the serial CPU baseline.

outlier counts cannot be calculated from it.

XII-A. Strong scaling

Strong scaling measures how runtime changes as more processing elements work on a fixed problem. For this workload, a fuller test would keep $N = 100{,}000$, $D = 16$, and $K = 10$ constant while measuring several MPI sizes, for example $P \in \{1, 2, 4, 8\}$. The current results contain only the serial baseline and one two-rank MPI measurement, so they do not establish a strong-scaling curve.

XII-B. Weak scaling

Weak scaling keeps the amount of work per rank approximately constant as more ranks are added. One way to test it is to increase N with P while keeping D, K, and the data-generation procedure fixed. Since the current chart uses one problem size, it provides no weak-scaling measurement.

XII-C. Limitations affecting performance

The timing includes per-iteration allocation and data transfers, so it measures more than kernel work. Global-memory atomics may also limit cluster accumulation, particularly when several MPI ranks share a GPU. Empty clusters are left unchanged. These costs should be kept in mind when reading the results.

XIII. Comparison with Existing Methods

The runtime table compares the serial implementation with two parallel variants under the same workload. Serial K-means is straightforward and avoids communication, but all distance calculations run on the CPU. MPI distributes those calculations across two ranks and reduces the mean runtime by 22.8%. The hybrid version records the lowest mean runtime, with a reduction of 49.2% against the serial baseline.

Prior work has also studied distributed-memory and GPU-based K-means [3], [4], [5]. A numerical comparison with those papers would require matching the processor and GPU models, dataset, cluster count, initialization, stopping condition, and timing method. The present results should therefore be read as a comparison among the three implementations in this project, not as a claim that they outperform published systems in general.

XIV. Conclusion

This project implements the Lloyd iteration of K-means in three forms: serial CPU, two-rank MPI CPU, and hybrid MPI+CUDA. The MPI program distributes points and combines local cluster statistics at each iteration. The hybrid version replaces the local CPU assignment with a CUDA kernel and uses

the CPU routine if a usable device is unavailable. A shared numerical interface makes it possible to compare the versions using the same input and validation checks.

For the workload of 100,000 points, 16 features, and 10 clusters, CUDA Hybrid records a mean runtime of 0.621 s, compared with 0.943 s for MPI CPU and 1.222 s for Serial CPU. These measurements correspond to mean speedups of about 1.97× for the hybrid version and 1.30× for MPI. The estimated efficiency of the two-rank MPI run is 64.8%. Additional tests at other rank counts and data sizes are needed before drawing broader conclusions about scalability.

XV. Future Scope

The next evaluation should retain the raw timing data and record the complete hardware and software configuration. Strong-scaling tests can hold the dataset fixed while varying the MPI rank count. Weak-scaling tests can increase the point count in proportion to the number of ranks. Each case should be repeated and reported with measures of spread, not only the mean and median.

Several code changes are also worth measuring. Reusing device buffers across iterations would avoid repeated allocation. Profiling could separate data-transfer time from kernel time, while block-level accumulation may reduce pressure on global atomics. The implementation could also add a policy for empty clusters and compare uniform sampling with K-means++ initialization. Each change should pass the existing SSE and centroid-matching checks before its timing results are considered.

References

[1] J. MacQueen. Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, pages 281–297, 1967.

[2] S. P. Lloyd. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.

[3] I. S. Dhillon and D. S. Modha. A data-clustering algorithm on distributed memory multiprocessors. In *Large-Scale Parallel Data Mining*, Lecture Notes in Computer Science, volume 1759, pages 245–260. Springer, 2000.

[4] R. Farivar, D. Rebolledo, E. Chan, and R. Campbell. A parallel implementation of K-means clustering on GPUs. In *Proceedings of the International Conference on Parallel and Distributed Processing Techniques and Applications (PDPTA)*, pages 340–345, 2008.

[5] Y. Li, K. Zhao, X. Chu, and J. Liu. Speeding up k-means algorithm by GPUs. *Journal of Computer and System Sciences*, 79(2):216–229, 2013.

[6] D. Arthur and S. Vassilvitskii. k-means++: The advantages of careful seeding. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1027–1035, 2007.

[7] Message Passing Interface Forum. *MPI: A Message-Passing Interface Standard, Version 4.1*, 2023.

[8] Y. LeCun, C. Cortes, and C. J. C. Burges. The MNIST database of handwritten digits. http://yann.lecun.com/exdb/mnist/.

[9] S. Stolfo, W. Fan, W. Lee, A. Prodromidis, and P. Chan. KDD Cup 1999 Data. UCI Machine Learning Repository, 1999.

[10] C. Meek, B. Thiesson, and D. Heckerman. US Census Data (1990). UCI Machine Learning Repository, 2001.

Plagiarism Check

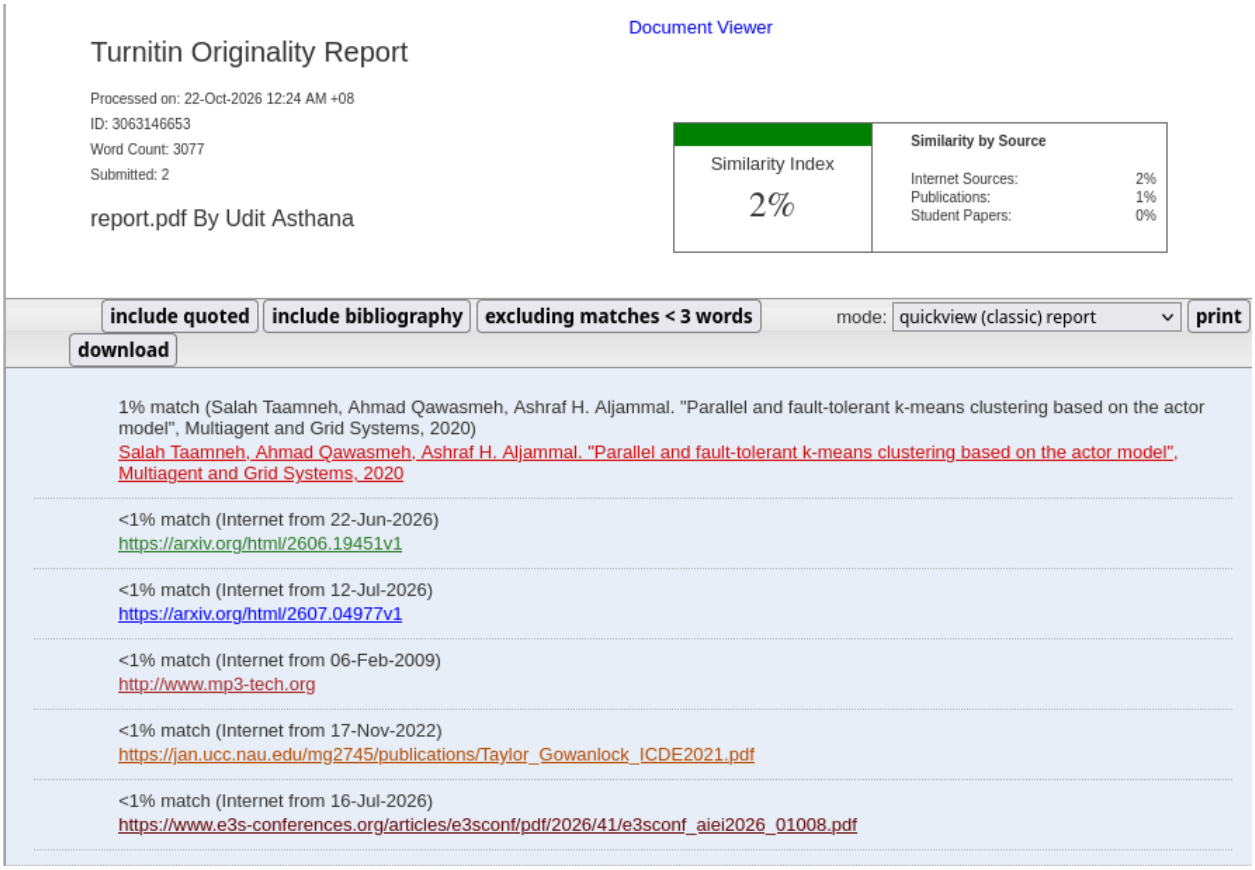


Fig. 2. The turnitin Plagiarism check. Shows 2% similarity.